

Contents

Practical 1 – Understanding Helm Architecture and Creating Your First Helm Chart	2
Practical 2 – Understanding Chart.yaml, values.yaml, and the Templates Directory	10
Practical 3 – Deploying Your First Helm Release	18
Practical 4 – Customizing Helm Deployments Using values.yaml and Command-Line Overrides.....	26
Practical 5 – Understanding Helm Templates and Built-in Objects.....	34
Practical 6 – Working with Helm Template Functions, Pipelines, Conditionals, and Loops	41
Practical 7 – Reusing Templates with Named Templates and _helpers.tpl.....	48

Practical 1 – Understanding Helm Architecture and Creating Your First Helm Chart

Overview

As Kubernetes adoption has grown, deploying applications has become increasingly complex. Even a simple web application may require multiple Kubernetes resource definitions such as Deployments, Services, ConfigMaps, Secrets, Ingresses, Persistent Volume Claims, and Horizontal Pod Autoscalers.

Consider a simple web application deployment.

deployment.yaml

service.yaml

configmap.yaml

secret.yaml

ingress.yaml

hpa.yaml

networkpolicy.yaml

Although each YAML file performs a different function, they collectively represent a single application.

Managing these files individually introduces several challenges:

- The number of YAML files increases as applications become more complex.
- Configuration values are often duplicated across multiple files.
- Updating applications requires modifying several manifests.
- Maintaining separate YAML files for Development, Testing, and Production environments leads to duplication.
- Versioning and application rollback become difficult.
- Sharing Kubernetes applications with other teams becomes cumbersome.

To solve these challenges, Helm introduces the concept of a **Chart**.

A Helm Chart is a package containing all the Kubernetes resource definitions required to deploy an application. Instead of managing numerous YAML files independently, Helm organizes them into a reusable and configurable package.

Helm is often referred to as the **Package Manager for Kubernetes**, just as:

Package Manager	Platform
apt	Debian / Ubuntu
dnf	Oracle Linux / RHEL
yum	Older Enterprise Linux distributions
pip	Python
npm	Node.js
Maven	Java
Helm	Kubernetes

Just as dnf install httpd installs the Apache web server on Oracle Linux, Helm installs complete applications on Kubernetes.

Prerequisites

Before beginning this practical, ensure that:

- Oracle Kubernetes Engine (OKE) or another Kubernetes cluster is running.
- kubectl is installed and configured.
- Helm version 3 is installed.
- You have access to the Kubernetes cluster.
- The current Kubernetes context is configured correctly.

What is a Helm Chart?

A Helm Chart is a collection of files that describe a Kubernetes application.

A chart contains:

- Kubernetes resource templates
- Configuration values
- Metadata
- Helper templates
- Optional dependencies

Think of a Helm Chart as the blueprint for an application.

Step-by-Step Procedure

Step 1 – Verify Kubernetes Connectivity

Before working with Helm, verify that your Kubernetes cluster is accessible.

Execute the following command.

kubectl get nodes

Explanation

The kubectl get nodes command displays all worker and control plane nodes that belong to the Kubernetes cluster.

This command confirms that:

- The Kubernetes cluster is running.
- Your kubeconfig file is configured correctly.
- kubectl can communicate with the Kubernetes API Server.
- Helm will be able to deploy applications to the cluster.

If the cluster is unreachable, Helm cannot install any applications because it relies on the Kubernetes API Server.

Expected Output

NAME	STATUS	ROLES	AGE	VERSION
oke-node-01	Ready	control-plane	15d	v1.32.x

Verification

Ensure that every node reports the **Ready** status.

If nodes display **NotReady**, resolve the Kubernetes issue before continuing.

Step 2 – Verify Helm Installation

Execute:

helm version

Explanation

This command displays the installed Helm version.

Unlike Helm Version 2, Helm Version 3 no longer requires Tiller.

Helm communicates directly with the Kubernetes API Server using your existing Kubernetes credentials.

Expected Output

```
version.BuildInfo{
  Version:"v3.x.x",
```

```
GitCommit:"...",  
GoVersion:"go1.xx"  
}
```

Verification

Confirm that the Helm version is displayed without any errors.

Step 3 – Display Helm Environment Information

Execute:

```
helm env
```

Explanation

The `helm env` command displays the environment variables used by Helm.

These variables determine where Helm stores:

- Repository cache
- Repository configuration
- Registry configuration
- Plugins
- Temporary files

Although administrators rarely modify these values, understanding where Helm stores its data is useful for troubleshooting.

Expected Output

```
HELM_CACHE_HOME=...
```

```
HELM_CONFIG_HOME=...
```

```
HELM_DATA_HOME=...
```

Verification

Verify that the environment paths are displayed successfully.

Step 4 – Create Your First Helm Chart

Execute the following command.

```
helm create employee-portal
```

Explanation

This is one of the most commonly used Helm commands.

The helm create command generates a complete Helm project with a predefined directory structure.

Rather than creating multiple Kubernetes YAML files manually, Helm scaffolds a production-ready chart containing the most commonly required resources.

Internally, Helm creates:

- Chart metadata
- Configuration values
- Deployment template
- Service template
- ServiceAccount template
- Ingress template
- Test template
- Helper template
- Charts directory for dependencies

No resources are created in Kubernetes at this stage.

The command only creates files on your local system.

Expected Output

```
Creating employee-portal
```

Verification

List the contents of your current directory.

```
ls
```

Expected output

```
employee-portal
```

Step 5 – Navigate to the Helm Chart Directory

Move into the newly created project.

```
cd employee-portal
```

Explanation

All Helm-related files are contained within this directory.

Every future operation, including modifying templates, updating configuration values, packaging the chart, and deploying applications, will be performed from this location.

Step 6 – Explore the Chart Structure

Display the directory structure.

```
tree .
```

If the tree utility is unavailable, execute:

```
find .
```

Expected Output

```
employee-portal/
```

```
Chart.yaml
```

```
values.yaml
```

```
charts/
```

```
templates/
```

```
templates/deployment.yaml
```

```
templates/service.yaml
```

```
templates/serviceaccount.yaml
```

templates/ingress.yaml

templates/NOTES.txt

templates/_helpers.tpl

templates/tests/

Explanation

Helm automatically generates a structured project.

Each file has a specific responsibility.

Rather than memorizing the directory, it is important to understand the purpose of every component.

The next practical explores each file in detail.

Step 7 – Examine the Generated Files

List the files individually.

ls -l

Explanation

You should observe the following important components.

File/Directory Purpose

Chart.yaml	Stores metadata about the chart
values.yaml	Stores configurable values used by templates
templates/	Contains Kubernetes resource templates
charts/	Stores dependent Helm charts
NOTES.txt	Displays post-installation instructions
_helpers.tpl	Stores reusable template helper functions

At this stage, you are not expected to understand every file completely.

The objective is to become familiar with the overall project layout.

Step 8 – Display Chart Information

Execute the following command.

```
helm show chart .
```

Explanation

The `helm show chart` command reads the `Chart.yaml` file and displays the chart metadata.

This command is useful when:

- Verifying chart details
- Inspecting downloaded charts
- Reviewing version information
- Confirming application metadata

Unlike opening the file manually, this command displays the metadata in a standardized format.

Expected Output

```
apiVersion: v2
```

```
name: employee-portal
```

```
description: A Helm chart for Kubernetes
```

```
type: application
```

```
version: 0.1.0
```

```
appVersion: "1.16.0"
```

Verification

Ensure that the displayed information matches the contents of the `Chart.yaml` file.

Practical 2 – Understanding Chart.yaml, values.yaml, and the Templates Directory

Objective

The objective of this practical is to understand the purpose of the important files generated by Helm. You will examine the Chart.yaml, values.yaml, and templates directory, modify chart metadata, update configuration values, and render Kubernetes manifests without deploying them to the cluster.

Learning Outcomes

After completing this practical, you will be able to:

- Understand the purpose of Chart.yaml.
- Modify chart metadata.
- Understand the purpose of values.yaml.
- Configure application settings using values.yaml.
- Explore the templates directory.
- Render Kubernetes manifests using Helm.
- Understand how Helm combines templates and values.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1 has been completed successfully.
- The employee-portal Helm chart exists.
- Helm and Kubernetes are configured correctly.

Step-by-Step Procedure

Step 1 – Navigate to the Helm Chart

If you are not already inside the chart directory, navigate to it.

```
cd employee-portal
```

Explanation

This command changes the current working directory to the Helm chart that was created in the previous practical.

Verification

```
pwd
```

The output should indicate that you are inside the employee-portal directory.

Step 2 – Examine the Chart Metadata

Open the Chart.yaml file.

```
cat Chart.yaml
```

or

```
vi Chart.yaml
```

Explanation

Chart.yaml is the metadata file for the Helm chart. It contains information that identifies the chart and provides version details.

A default file appears similar to:

```
apiVersion: v2
```

```
name: employee-portal
```

```
description: A Helm chart for Kubernetes
```

```
type: application
```

```
version: 0.1.0
```

```
appVersion: "1.16.0"
```

Each field has a specific purpose.

Field	Description
apiVersion	Helm chart specification version.
name	Name of the chart.
description	Brief description of the application.
type	Indicates whether the chart is an application or a library.
version	Version of the Helm chart.
appVersion	Version of the application being deployed.

Verification

Display only the chart name.

```
grep name Chart.yaml
```

Expected output:

```
name: employee-portal
```

Step 3 – Modify the Chart Metadata

Edit the Chart.yaml file.

```
vi Chart.yaml
```

Replace the existing values with the following:

```
apiVersion: v2
```

```
name: employee-portal
```

```
description: Employee Management Portal Helm Chart
```

```
type: application
```

```
version: 1.0.0
```

```
appVersion: "1.0"
```

Save the file.

Explanation

Updating the metadata helps identify the application and the version of the chart.

The **chart version** and **application version** are independent.

- Increase the **chart version** when the chart structure or templates change.
- Increase the **application version** when a new version of the application is released.

Verification

Display the updated chart information.

```
helm show chart .
```

Verify that the updated values are displayed.

Step 4 – Explore the Default Configuration File

Open the values.yaml file.

```
cat values.yaml
```

or

```
vi values.yaml
```

Explanation

The values.yaml file stores the default configuration values used by the templates.

Instead of modifying Kubernetes YAML files directly, you update values in this file.

Some commonly used parameters include:

```
replicaCount: 1
```

```
image:
```

```
  repository: nginx
```

```
  pullPolicy: IfNotPresent
```

```
  tag: ""
```

```
service:
```

```
  type: ClusterIP
```

```
  port: 80
```

These values are automatically used by the templates during deployment.

Verification

Display the replica count.

```
grep replicaCount values.yaml
```

Expected output:

```
replicaCount: 1
```

Step 5 – Modify the Default Configuration

Edit the values.yaml file.

```
vi values.yaml
```

Change the following values.

```
replicaCount: 3
```

Locate the Service section and change:

service:

```
type: NodePort
```

Save the file.

Explanation

These values will not immediately modify Kubernetes resources.

Instead, they become the new default configuration used whenever the chart is installed or upgraded.

Verification

Display the updated values.

```
grep replicaCount values.yaml
```

```
grep -A2 service values.yaml
```

Expected output:

```
replicaCount: 3
```

service:

```
type: NodePort
```

Step 6 – Explore the Templates Directory

List the template files.

```
ls templates
```

Expected output:

```
NOTES.txt
```

```
_helpers.tpl
```

```
deployment.yaml
```

```
service.yaml
```

```
serviceaccount.yaml
```

```
ingress.yaml
```

```
tests
```

Explanation

The templates directory contains Kubernetes resource templates.

Unlike normal Kubernetes YAML files, these templates contain placeholders that Helm replaces using values from the values.yaml file.

Each template creates a different Kubernetes resource.

Template	Kubernetes Resource
deployment.yaml	Deployment
service.yaml	Service
ingress.yaml	Ingress
serviceaccount.yaml	ServiceAccount
NOTES.txt	Installation notes
tests	Test resources

Verification

Display the deployment template.

```
cat templates/deployment.yaml
```

Observe the placeholders enclosed in double curly braces ({{ }}).

Step 7 – Locate Template Variables

Search for the replica count inside the Deployment template.

```
grep replicaCount templates/deployment.yaml
```

Expected output:

```
replicas: {{ .Values.replicaCount }}
```

Explanation

The expression:

```
{{ .Values.replicaCount }}
```

instructs Helm to retrieve the value of replicaCount from the values.yaml file.

If the value is changed in values.yaml, Helm automatically updates the Deployment during rendering.

Verification

Locate the image repository.

```
grep repository templates/deployment.yaml
```

Observe that the image is also retrieved from values.yaml.

Step 8 – Render the Templates

Generate the Kubernetes manifests without deploying them.

```
helm template employee-demo .
```

Explanation

The helm template command processes all templates using the values from values.yaml.

Instead of sending the manifests to Kubernetes, Helm prints the generated YAML to the terminal.

This allows administrators to validate the generated resources before deployment.

Verification

Observe the generated Deployment.

Verify that:

- The replica count is **3**.
- The Service type is **NodePort**.
- The chart metadata is reflected in the generated resources.

Step 9 – Save the Rendered Output

Redirect the rendered manifests to a file.

```
helm template employee-demo . > rendered-output.yaml
```

Explanation

This command saves all generated Kubernetes manifests into a single file.

Administrators commonly use this approach to review or archive the generated resources before deployment.

Verification

Display the generated file.

```
head rendered-output.yaml
```

Verify that the file contains standard Kubernetes YAML manifests.

Best Practices

- Keep chart metadata accurate and up to date.
- Store all configurable settings in values.yaml.
- Avoid modifying templates for environment-specific values.
- Use helm template to validate charts before deployment.
- Maintain semantic versioning for chart versions.

Practical 3 – Deploying Your First Helm Release

Objective

The objective of this practical is to deploy a Helm chart into a Kubernetes cluster by creating a Helm Release. You will learn how to install a chart, verify the deployed resources, inspect release information, view release history, and uninstall the application.

Learning Outcomes

After completing this practical, you will be able to:

- Install a Helm chart.
- Understand the concept of a Helm Release.
- Verify deployed Kubernetes resources.
- View release information.
- Display release history.
- Uninstall a Helm Release.
- Verify that resources are removed from the cluster.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1 and Practical 2 have been completed successfully.
- The employee-portal chart is available.
- Kubernetes cluster is running.
- Helm is installed and configured.

Step-by-Step Procedure

Step 1 – Verify the Helm Chart

Before deploying the application, verify that Helm recognizes the chart.

Execute the following command.

```
helm show chart .
```

Explanation

The helm show chart command displays the metadata stored in the Chart.yaml file. This allows you to confirm that the chart information is correct before deployment.

Verification

Ensure that the following information is displayed:

- Chart Name
- Description
- Version
- Application Version

Step 2 – Validate the Chart

Validate the Helm chart before deployment.

helm lint .

Explanation

The helm lint command checks the chart for common errors such as:

- Missing required files
- Invalid YAML syntax
- Template issues
- Incorrect chart metadata

Validating the chart before deployment helps identify problems early.

Expected Output

==> Linting .

[INFO] Chart.yaml: icon is recommended

1 chart(s) linted, 0 chart(s) failed

Verification

Ensure that no chart failures are reported.

Step 3 – Install the Helm Chart

Deploy the chart into Kubernetes.

```
helm install employee-app .
```

Explanation

The helm install command creates a new **Helm Release** using the current chart.

Command Breakdown

Component	Description
helm	Executes the Helm CLI
install	Creates a new release
employee-app	Release name
.	Uses the current directory as the chart

Unlike Kubernetes YAML files, the chart itself is **not** deployed.

Helm performs the following actions internally:

1. Reads Chart.yaml.
2. Reads values.yaml.
3. Reads every file in the templates directory.
4. Replaces template variables.
5. Generates Kubernetes manifests.
6. Sends the manifests to the Kubernetes API Server.
7. Stores release information.

Expected Output

NAME: employee-app

LAST DEPLOYED: ...

NAMESPACE: default

STATUS: deployed

REVISION: 1

Verification

Check whether the release has been installed.

```
helm list
```

Expected Output

NAME	NAMESPACE	REVISION	STATUS	CHART
employee-app	default	1	deployed	employee-portal-1.0.0

Step 4 – Verify the Kubernetes Resources

Display all Kubernetes resources.

```
kubectl get all
```

Explanation

This command displays every resource created by Helm.

You should observe:

- Deployment
- ReplicaSet
- Pods
- Service

Helm creates these resources based on the templates contained within the chart.

Verification

Ensure that all Pods are in the **Running** state.

Step 5 – View the Pods

Display the Pods.

```
kubectl get pods
```

Explanation

This command lists all Pods created by the Deployment.

If the replica count in values.yaml was changed to **3**, three Pods should be running.

Verification

Expected Output

NAME	READY	STATUS
employee-app-xxxxxxxxx-abcde	1/1	Running
employee-app-xxxxxxxxx-fghij	1/1	Running
employee-app-xxxxxxxxx-klmno	1/1	Running

Step 6 – Display the Deployment

View the Deployment resource.

```
kubectl get deployment
```

Explanation

The Deployment manages the ReplicaSet and ensures that the desired number of Pods are always running.

Verification

Confirm that the number of READY replicas matches the desired replicas.

Step 7 – Display the Service

Display the Service.

```
kubectl get svc
```

Explanation

The Service exposes the application within the Kubernetes cluster.

If you modified the Service type to **NodePort** in the previous practical, the NodePort should be displayed.

Verification

Verify:

- Service Name
- Service Type
- Cluster IP
- Port
- NodePort (if applicable)

Step 8 – View Detailed Release Information

Display detailed information about the release.

```
helm status employee-app
```

Explanation

The helm status command displays:

- Release status
- Namespace
- Revision
- Resources
- Deployment notes

This command is commonly used by administrators to verify application deployment.

Verification

Ensure that the release status is **deployed**.

Step 9 – Display Release History

View the release history.

```
helm history employee-app
```

Explanation

Every installation and upgrade creates a new revision.

The history command displays all revisions associated with the release.

Expected Output

```
REVISION  STATUS  CHART
1         deployed employee-portal-1.0.0
```

Verification

Ensure that Revision **1** is displayed.

Step 10 – Display the Values Used During Installation

Display the values used by the release.

```
helm get values employee-app
```

Explanation

This command retrieves the configuration values that Helm used during installation.

Administrators frequently use this command to verify the release configuration.

Verification

Confirm that the values displayed match the contents of the values.yaml file.

Step 11 – Display the Generated Kubernetes Manifests

Display the manifests generated during installation.

```
helm get manifest employee-app
```

Explanation

This command retrieves the Kubernetes manifests that Helm rendered and deployed.

Unlike `helm template`, this command displays the manifests associated with an existing release.

Verification

Ensure that the output contains:

- Deployment
- Service
- ServiceAccount
- Other generated resources

Step 12 – Uninstall the Helm Release

Remove the deployed application.

```
helm uninstall employee-app
```

Explanation

The `helm uninstall` command removes the release and all Kubernetes resources associated with it.

Helm deletes:

- Deployment
- ReplicaSet
- Pods
- Service
- ServiceAccount
- Other managed resources

Expected Output

release "employee-app" uninstalled

Verification

Display the list of releases.

helm list

No releases should be displayed.

Step 13 – Verify Resource Removal

Verify that Kubernetes resources have been removed.

kubectl get all

Explanation

Since the release has been uninstalled, the resources managed by Helm should no longer exist.

Verification

The application resources should not appear in the output.

Practical 4 – Customizing Helm Deployments Using values.yaml and Command-Line Overrides

Objective

The objective of this practical is to learn how to customize Helm deployments without modifying the Kubernetes templates. You will update the values.yaml file, override configuration values from the command line, use custom values files, and verify that the changes are reflected in the deployed application.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1, 2, and 3 have been completed.
- The employee-portal Helm chart is available.
- Kubernetes cluster is running.
- Helm is configured successfully.

Step-by-Step Procedure

Step 1 – Review the Current Configuration

Display the default configuration.

```
cat values.yaml
```

Explanation

The values.yaml file contains the default values that are used whenever the chart is installed.

Typical values include:

- Replica count
- Container image
- Service type
- Service port
- Resource limits

Review these values before making any modifications.

Verification

Confirm that values similar to the following exist:

```
replicaCount: 3
```

```
service:
```

```
  type: NodePort
```

Step 2 – Change the Number of Replicas

Edit the values.yaml file.

```
vi values.yaml
```

Locate the following section.

```
replicaCount: 3
```

Change it to:

```
replicaCount: 5
```

Save the file.

Explanation

The replicaCount parameter specifies how many Pod replicas should be created by the Deployment.

Increasing this value allows the application to handle more requests and provides better availability.

Verification

Display the updated value.

```
grep replicaCount values.yaml
```

Expected output:

```
replicaCount: 5
```

Step 3 – Upgrade the Existing Release

Apply the new configuration.

```
helm upgrade employee-app .
```

Explanation

The helm upgrade command updates an existing release using the latest chart and configuration.

Command Breakdown

Component	Description
helm	Executes the Helm CLI
upgrade	Updates an existing release
employee-app	Name of the existing release
.	Uses the current chart

Helm performs the following operations:

1. Reads the updated chart.
2. Reads the modified values.yaml.
3. Generates new Kubernetes manifests.
4. Compares the new manifests with the existing deployment.
5. Applies only the required changes.

Expected Output

Release "employee-app" has been upgraded.

Verification

Display the Deployment.

```
kubect! get deployment
```

The desired replicas should now be **5**.

Step 4 – Verify the Running Pods

Display the Pods.

```
kubect! get pods
```

Explanation

Since the Deployment now requires five replicas, Kubernetes automatically creates additional Pods.

Verification

Five Pods should be displayed.

Step 5 – Change the Service Type

Edit the configuration.

```
vi values.yaml
```

Locate:

```
service:
```

```
  type: NodePort
```

Change it to:

```
service:
```

```
  type: ClusterIP
```

Save the file.

Explanation

The Service type determines how the application is exposed.

Service Type	Description
ClusterIP	Internal access within the cluster
NodePort	External access through a node port
LoadBalancer	External access using a cloud load balancer

Verification

Display the updated configuration.

```
grep -A2 service values.yaml
```

Step 6 – Upgrade the Release Again

Apply the changes.

```
helm upgrade employee-app .
```

Explanation

Helm updates only the Service resource because only the Service configuration has changed.

Verification

Display the Service.

```
kubectl get svc
```

Verify that the Service type is now **ClusterIP**.

Step 7 – Override Values Using the Command Line

Instead of editing the values.yaml file, override the replica count during the upgrade.

```
helm upgrade employee-app . --set replicaCount=2
```

Explanation

The --set option allows you to override one or more values directly from the command line.

This method is useful when testing small configuration changes without modifying the values file.

Verification

Display the Deployment.

```
kubectl get deployment
```

Verify that the desired replica count is **2**.

Step 8 – Override Multiple Values

Update multiple settings in a single command.

```
helm upgrade employee-app . \
```

```
--set replicaCount=4 \
```

```
--set service.type=NodePort
```

Explanation

Multiple --set options can be used to override several configuration values during installation or upgrade.

Verification

Display the Service.

```
kubectl get svc
```

Display the Deployment.

```
kubectl get deployment
```

Verify:

- Replica count is **4**.
- Service type is **NodePort**.

Step 9 – Create a Custom Values File

Create a new file named dev-values.yaml.

vi dev-values.yaml

Add the following content.

```
replicaCount: 2
```

```
service:
```

```
  type: NodePort
```

Save the file.

Explanation

Custom values files allow different environments to use different configurations without changing the default values.yaml.

Example:

- Development
- Testing
- Production

Each environment can have its own values file.

Verification

Display the file.

```
cat dev-values.yaml
```

Step 10 – Deploy Using a Custom Values File

Upgrade the release using the new configuration.

```
helm upgrade employee-app . -f dev-values.yaml
```

Explanation

The -f option tells Helm to use the specified values file instead of the default configuration.

Verification

Display the Deployment.

```
kubectl get deployment
```

Verify that the configuration matches the values defined in dev-values.yaml.

Step 11 – View the Applied Values

Display the values used by the release.

```
helm get values employee-app
```

Explanation

This command displays the configuration currently associated with the release.

It helps administrators verify which values were applied during installation or upgrade.

Verification

Ensure that the displayed values match the custom values file.

Step 12 – Compare Default and Custom Values

Display both configuration files.

```
cat values.yaml
```

```
cat dev-values.yaml
```

Explanation

Comparing these files demonstrates how different environments can maintain separate configurations while sharing the same Helm chart.

Best Practices

- Keep the default values.yaml file generic.
- Create separate values files for Development, Testing, and Production.
- Use --set only for temporary overrides.
- Store environment-specific values in version control.

- Review changes before upgrading a release.

Practical 5 – Understanding Helm Templates and Built-in Objects

Objective

The objective of this practical is to understand how Helm templates work, how values are injected into Kubernetes manifests, and how to use Helm built-in objects such as `.Values`, `.Release`, and `.Chart`. You will modify templates, render them, and observe how Helm generates Kubernetes manifests dynamically.

Learning Outcomes

After completing this practical, you will be able to:

- Understand Helm template syntax.
- Use the `.Values` object.
- Use the `.Release` object.
- Use the `.Chart` object.
- Render templates using Helm.
- Verify dynamically generated Kubernetes manifests.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1–4 have been completed.
- The `employee-portal` chart is available.
- Helm and Kubernetes are configured correctly.

Step-by-Step Procedure

Step 1 – Examine the Deployment Template

Open the Deployment template.

```
cat templates/deployment.yaml
```

or

```
vi templates/deployment.yaml
```

Explanation

The deployment.yaml file is not a standard Kubernetes manifest. It is a **Helm template** that contains placeholders enclosed within double curly braces `{{ }}`.

Example:

```
replicas: {{ .Values.replicaCount }}
```

Instead of using a fixed value, Helm replaces this placeholder with the value stored in values.yaml during chart rendering.

Verification

Search for template expressions.

```
grep "{{" templates/deployment.yaml
```

Observe multiple template expressions in the file.

Step 2 – Understand the .Values Object

Locate the following line.

```
replicas: {{ .Values.replicaCount }}
```

Explanation

The .Values object provides access to all values defined in the values.yaml file.

For example, if the following exists in values.yaml:

```
replicaCount: 3
```

Helm renders the Deployment as:

```
replicas: 3
```

This allows the same template to generate different manifests by simply changing the values file.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the rendered Deployment contains:

```
replicas: 3
```

(or the current value configured in your values.yaml file).

Step 3 – Use the .Release Object

Search for the Release object.

```
grep ".Release" templates/deployment.yaml
```

If it is not present, edit the Deployment labels.

```
vi templates/deployment.yaml
```

Add the following label under metadata.labels.

```
release: {{ .Release.Name }}
```

Save the file.

Explanation

The .Release object contains information about the Helm release.

Some commonly used properties are:

Object	Description
.Release.Name	Name of the Helm release
.Release.Namespace	Kubernetes namespace
.Release.Revision	Current release revision
.Release.Service	Deployment tool (Helm)

These values are automatically populated by Helm during installation.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the label appears as:

```
release: employee-demo
```

Step 4 – Use the .Chart Object

Edit the Deployment template.

```
vi templates/deployment.yaml
```

Add the following label.

```
chart: {{ .Chart.Name }}
```

Add another label.

```
version: {{ .Chart.Version }}
```

Save the file.

Explanation

The `.Chart` object retrieves information from the `Chart.yaml` file.

For example:

```
name: employee-portal
```

```
version: 1.0.0
```

Helm automatically substitutes these values when rendering the chart.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the labels display:

```
chart: employee-portal
```

```
version: 1.0.0
```

Step 5 – Display All Rendered Resources

Render the complete chart.

```
helm template employee-demo .
```

Explanation

The `helm template` command processes every template in the `templates` directory and replaces all template expressions with actual values.

No resources are deployed to Kubernetes.

This command is useful for validating templates before installation.

Verification

Confirm that the output includes:

- Deployment

- Service
- ServiceAccount
- Ingress (if enabled)
- Test Pod

Step 6 – Save the Rendered Output

Generate the manifests into a file.

```
helm template employee-demo . > output.yaml
```

Explanation

Redirecting the output to a file allows you to inspect the generated Kubernetes manifests before deployment.

This technique is commonly used during development and troubleshooting.

Verification

Display the beginning of the generated file.

```
head output.yaml
```

Ensure the file contains valid Kubernetes YAML.

Step 7 – Compare Template and Rendered Manifest

Display the template.

```
cat templates/deployment.yaml
```

Display the rendered manifest.

```
cat output.yaml
```

Explanation

Compare the two files.

The template contains Helm expressions such as:

```
{{ .Values.replicaCount }}
```

The rendered manifest contains actual values such as:

```
replicas: 3
```

This demonstrates how Helm converts templates into standard Kubernetes manifests.

Step 8 – Override Values During Rendering

Render the chart with a different replica count.

```
helm template employee-demo . --set replicaCount=6
```

Explanation

The `--set` option temporarily overrides values without modifying the `values.yaml` file.

Only the rendered output changes; the chart files remain unchanged.

Verification

Verify that the Deployment contains:

```
replicas: 6
```

Step 9 – Render Using a Custom Values File

Render the chart using the development configuration.

```
helm template employee-demo . -f dev-values.yaml
```

Explanation

The `-f` option instructs Helm to use the specified values file during rendering.

This allows administrators to preview manifests for different environments without deploying them.

Verification

Verify that the generated manifests reflect the values defined in `dev-values.yaml`.

Step 10 – Validate the Generated YAML

Save the rendered output.

```
helm template employee-demo . > rendered.yaml
```

Display the file.

```
cat rendered.yaml
```

Explanation

Review the generated YAML and ensure that:

- Resource names are correct.
- Labels are populated.
- Replica count is correct.
- Service type matches the configuration.
- Image information is accurate.

This validation step helps detect configuration issues before deployment.

Best Practices

- Store all configurable values in values.yaml.
- Avoid hardcoding values inside templates.
- Use Helm built-in objects for metadata.
- Validate templates using helm template before deploying.
- Keep templates simple and readable.

Practical 6 – Working with Helm Template Functions, Pipelines, Conditionals, and Loops

Objective

The objective of this practical is to learn how to use Helm template functions, pipelines, conditional statements, and loops to build dynamic Helm charts. You will modify templates to render resources based on configuration values and generate multiple Kubernetes objects dynamically.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1–5 have been completed.
- The employee-portal chart is available.
- Helm and Kubernetes are configured correctly.

Step-by-Step Procedure

Step 1 – Create a New ConfigMap Template

Create a new template.

```
vi templates/app-config.yaml
```

Add the following content.

```
apiVersion: v1
```

```
kind: ConfigMap
```

```
metadata:
```

```
  name: app-config
```

```
data:
```

```
  application: "{{ .Chart.Name }}"
```

```
  environment: "{{ .Values.environment }}"
```

Save the file.

Explanation

This template creates a ConfigMap that retrieves:

- The application name from the Chart.yaml file.
- The environment value from the values.yaml file.

Verification

Add the following value to values.yaml.

```
environment: development
```

Render the chart.

```
helm template employee-demo .
```

Verify that the generated ConfigMap contains:

data:

```
application: employee-portal
```

```
environment: development
```

Step 2 – Use the default Function

Modify the ConfigMap.

```
vi templates/app-config.yaml
```

Replace the environment line with:

```
environment: "{{ default \"development\" .Values.environment }}"
```

Save the file.

Explanation

The default function assigns a fallback value when the specified variable is not defined.

Syntax:

```
default <default-value> <variable>
```

If environment is removed from values.yaml, Helm automatically uses **development**.

Verification

Remove the environment value from values.yaml.

Render the chart.

```
helm template employee-demo .
```

Verify that the ConfigMap displays:

environment: development

Step 3 – Use the upper Function

Modify the template.

vi templates/app-config.yaml

Update the environment line.

environment: "{{ upper .Values.environment }}"

Save the file.

Explanation

The upper function converts text to uppercase.

Example:

development

becomes

DEVELOPMENT

Verification

Add the following value.

environment: production

Render the chart.

helm template employee-demo .

Verify that the output contains:

environment: PRODUCTION

Step 4 – Use Pipelines

Modify the template.

vi templates/app-config.yaml

Replace the previous line with:

environment: "{{ .Values.environment | upper }}"

Save the file.

Explanation

A pipeline passes the output of one function as the input to another.

The following two statements produce the same result:

```
upper .Values.environment
```

and

```
.Values.environment | upper
```

Pipelines improve readability when multiple functions are chained together.

Verification

Render the chart.

```
helm template employee-demo .
```

Confirm that the value is displayed in uppercase.

Step 5 – Create a Conditional Resource

Open the Service template.

```
vi templates/service.yaml
```

Wrap the entire file inside the following condition.

```
{{- if .Values.service.enabled }}
```

<Service Template>

```
{{- end }}
```

Add the following value to values.yaml.

```
service:
```

```
  enabled: true
```

Save both files.

Explanation

The if statement determines whether a resource should be generated.

If the condition evaluates to **true**, the Service manifest is rendered.

If the condition evaluates to **false**, Helm skips the resource.

Verification

Render the chart.

helm template employee-demo .

Verify that the Service resource is displayed.

Step 6 – Disable the Service

Edit the values file.

vi values.yaml

Modify the value.

service:

enabled: false

Save the file.

Explanation

When the condition evaluates to **false**, Helm does not generate the Service manifest.

Verification

Render the chart.

helm template employee-demo .

Verify that the Service resource is no longer present in the output.

Step 7 – Create a List of Allowed Environments

Edit values.yaml.

vi values.yaml

Add the following section.

environments:

- development
- testing
- production

Save the file.

Explanation

This creates a list that can be processed using a loop.

Step 8 – Generate Multiple ConfigMaps Using range

Create a new template.

```
vi templates/environment-configmaps.yaml
```

Add the following content.

```
{{- range .Values.environments }}
```

```
  apiVersion: v1
```

```
  kind: ConfigMap
```

```
  metadata:
```

```
    name: {{ . }}-config
```

```
  data:
```

```
    environment: "{{ . }}"
```

```
---
```

```
{{- end }}
```

Save the file.

Explanation

The range statement iterates through every item in the list.

For each environment, Helm generates a separate ConfigMap.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that three ConfigMaps are generated.

Example:

```
development-config
```

testing-config

production-config

Step 9 – Count the Generated Resources

Save the rendered output.

```
helm template employee-demo . > output.yaml
```

Display all ConfigMaps.

```
grep "kind: ConfigMap" output.yaml
```

Explanation

This verifies that Helm created multiple resources from a single template using a loop.

Verification

Ensure that three ConfigMaps are displayed.

Step 10 – Deploy the Updated Chart

Install or upgrade the release.

```
helm upgrade employee-app .
```

Explanation

Helm renders all templates and deploys the generated resources to Kubernetes.

The Deployment now includes dynamically generated ConfigMaps based on the values file.

Verification

Display the ConfigMaps.

```
kubectl get configmaps
```

Verify that the generated ConfigMaps exist in the cluster.

Practical 7 – Reusing Templates with Named Templates and _helpers.tpl

Objective

The objective of this practical is to learn how to create reusable template components using **Named Templates** and the `_helpers.tpl` file. You will define reusable template blocks, include them in Kubernetes resource templates, and understand how Helm helps eliminate duplicate code across multiple templates.

Prerequisites

Before beginning this practical, ensure that:

- Practical 1 through Practical 6 have been completed successfully.
- The employee-portal Helm chart is available.
- The employee-app release is deployed successfully.
- Kubernetes cluster is running.
- Helm is installed and configured.
- Verify the chart is valid.

`helm lint .`

Expected Output

1 chart(s) linted, 0 chart(s) failed

Step-by-Step Procedure

Step 1 – Examine the `_helpers.tpl` File

Open the helper template file.

```
cat templates/_helpers.tpl
```

or

```
vi templates/_helpers.tpl
```

Explanation

The `_helpers.tpl` file contains reusable template definitions that can be referenced by multiple resource templates.

Instead of repeating the same labels, names, or annotations in every YAML file, Helm allows you to define them once and reuse them throughout the chart.

This improves consistency and simplifies maintenance.

Verification

Observe that the file already contains helper definitions generated by Helm.

Step 2 – Create a Named Template

Edit the `_helpers.tpl` file.

```
vi templates/_helpers.tpl
```

Append the following template to the end of the file.

```
{{- define "employee.labels" }}  
app.kubernetes.io/name: {{ .Chart.Name }}  
app.kubernetes.io/version: {{ .Chart.Version }}  
app.kubernetes.io/instance: {{ .Release.Name }}  
app.kubernetes.io/managed-by: {{ .Release.Service }}  
{{- end }}
```

Save the file.

Explanation

The define function creates a reusable template named **employee.labels**.

This template defines a common set of labels that can be included in any Kubernetes resource.

Verification

Ensure the template has been saved successfully.

Step 3 – Include the Named Template in the Deployment

Open the Deployment template.

```
vi templates/deployment.yaml
```

Locate the `metadata.labels` section.

Replace the existing labels with:

labels:

```
{{ include "employee.labels" . | indent 4 }}
```

Save the file.

Explanation

The include function inserts the contents of the named template into the Deployment.

The indent 4 function ensures the generated YAML is properly aligned under the labels section.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the generated Deployment contains the labels defined in _helpers.tpl.

Step 4 – Reuse the Named Template in the Service

Open the Service template.

```
vi templates/service.yaml
```

Locate the metadata.labels section.

Replace the labels with:

labels:

```
{{ include "employee.labels" . | indent 4 }}
```

Save the file.

Explanation

The same helper template is now reused in the Service resource.

This demonstrates how a single template definition can be shared across multiple Kubernetes manifests.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that both the Deployment and Service contain identical labels.

Step 5 – Use the Helper in the ConfigMap

Open the ConfigMap template created in the previous practical.

```
vi templates/app-config.yaml
```

Add the following section.

```
metadata:
```

```
  name: app-config
```

```
  labels:
```

```
{{ include "employee.labels" . | indent 4 }}
```

Save the file.

Explanation

The ConfigMap now uses the same reusable labels, ensuring consistency across all resources.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the ConfigMap contains the same labels.

Step 6 – Validate the Chart

Run the Helm linter.

```
helm lint .
```

Explanation

The linter validates the chart after the helper template has been introduced.

This helps detect syntax errors or indentation issues.

Verification

Expected Output

```
1 chart(s) linted, 0 chart(s) failed
```

Step 7 – Upgrade the Existing Release

Deploy the updated chart.

```
helm upgrade employee-app .
```

Explanation

The release is updated with the modified templates that now use reusable helper functions.

Helm updates only the resources whose manifests have changed.

Verification

Expected Output

```
Release "employee-app" has been upgraded
```

Step 8 – Verify the Deployment Labels

Display the Deployment.

```
kubectl get deployment employee-app -o yaml
```

Explanation

This command displays the complete Deployment manifest currently running in the cluster.

Verification

Locate the labels section.

Verify that it contains:

labels:

```
app.kubernetes.io/name: employee-portal
```

```
app.kubernetes.io/version: "1.0.0"
```

```
app.kubernetes.io/instance: employee-app
```

```
app.kubernetes.io/managed-by: Helm
```

Step 9 – Verify the Service Labels

Display the Service.

```
kubectl get svc employee-app -o yaml
```

Explanation

The Service should use the same labels because it references the same helper template.

Verification

Confirm that the labels match those displayed in the Deployment.

Step 10 – Modify the Helper Template

Edit the helper template.

```
vi templates/_helpers.tpl
```

Add the following label.

```
environment: production
```

The helper should now resemble:

```
{{- define "employee.labels" }}  
app.kubernetes.io/name: {{ .Chart.Name }}  
app.kubernetes.io/version: {{ .Chart.Version }}  
app.kubernetes.io/instance: {{ .Release.Name }}  
app.kubernetes.io/managed-by: {{ .Release.Service }}  
environment: production  
{{- end }}
```

Save the file.

Explanation

Instead of updating multiple resource templates individually, you modify the helper template once.

All templates that include this helper automatically receive the updated label.

Verification

Render the chart.

```
helm template employee-demo .
```

Verify that the new environment label appears in all generated resources.

Step 11 – Upgrade the Release

Apply the updated helper template.

```
helm upgrade employee-app .
```

Explanation

Helm regenerates the manifests using the updated helper template and applies the changes to the cluster.

Verification

Display all resources.

```
kubectl get all --show-labels
```

Verify that the environment=production label appears on the Deployment, Service, and other resources that use the helper template.

Best Practices

- Store reusable labels and annotations in `_helpers.tpl`.
- Avoid duplicating metadata across multiple templates.
- Use descriptive names for helper templates.
- Validate templates using `helm lint` after making changes.
- Use helper templates to maintain consistency across resources.

FOR PARTICIPANTS

Packaging and Managing Helm Charts

The objective of the practical is to package a Helm chart into a distributable archive, inspect chart information, verify chart integrity, and install a packaged chart.

Tasks

- Validate a Helm chart.
- Package a Helm chart.
- Inspect packaged charts.
- Install a chart from a package.
- Update chart versions.
- Verify packaged chart contents.
- Understand the chart packaging workflow.